

INTEGRATED MACHINE LEARNING TEXTBOOK

예시로 이해하고 프로젝트로 완성하는 머신러닝

학습 방식의 큰 그림부터 회귀·분류 이론, 성능지표, 라이브러리 활용, 전처리·평가·저장까지 연결한 통합 입문 교재

강의 PDF · regression.ipynb · 기존 교재 4종 통합
오류 교정 및 누락 개념 보강판

통합 교재의 학습 지도

개념을 이해한 뒤 같은 개념이 코드와 프로젝트에서 어디에 나타나는지 연결합니다.

1부	AI와 머신러닝, 지도·비지도·강화학습
2부	환경, 데이터, 전처리와 공통 프로젝트 흐름
3부	회귀: 선형·다중·다항·SGD·규제와 평가
4부	분류: 로지스틱·KNN·트리·앙상블·텍스트·SVM
5부	Loss, 성능지표, Bagging·Boosting
6부	모델 개선, 저장, 새 데이터 예측과 재현성
7부	라이브러리·메서드와 그래프 표현 부록

학습 방법

각 장에서 **일상 예시** → **데이터의 X와 y** → **모델의 판단** → **코드** → **평가** → **주의점** 순서로 읽습니다.

AI, 머신러닝, 딥러닝의 관계

인공지능 AI

사람의 판단·인식·문제 해결을 컴퓨터로 구현하려는 가장 넓은 영역입니다.

머신러닝 ML

규칙을 사람이 모두 작성하지 않고 데이터에서 패턴을 학습합니다.

딥러닝 DL

여러 층의 신경망을 사용하며 이미지·음성·텍스트에 강합니다.

스팸 판별

사람이 "무료라는 단어가 있으면 스팸" 규칙을 전부 작성하면 규칙 기반 AI입니다. 과거 정상·스팸 메일에서 규칙을 배우면 머신러닝입니다. 문장 표현을 신경망이 직접 학습하면 딥러닝 접근입니다.

핵심

딥러닝은 머신러닝의 일부이고, 머신러닝은 인공지능을 구현하는 방법 중 하나입니다.

지도·비지도·강화학습 비교

방식	정답	모델이 배우는 것	예시
지도학습	있음	X에서 y를 예측하는 규칙	집값, 합격, 스팸
비지도학습	없음	데이터 안의 구조와 비슷한 그룹	고객 군집, 이상 탐지
강화학습	정답표 대신 보상	상태에서 어떤 행동이 장기 보상을 높이는가	게임, 로봇 제어

구분 주의
비지도학습의 군집화도 데이터를 “나누지만”, 이미 정해진 정답 클래스를 맞추는 분류와는 다릅니다.

지도학습: 문제와 정답을 함께 제공

X 입력	y 정답	문제
면적, 층수, 역 거리	집값 5억 2천만원	회귀
메일 단어와 발신 정보	스팸	분류

학습 데이터에서는 X와 y를 함께 보여주고, 시험과 실제 사용에서는 새로운 X만 넣어 y를 예측합니다.



일반화

학습 문제를 외우는 것이 아니라 처음 보는 문제에도 적용되는 규칙을 찾는 것이 목표입니다.

비지도학습: 정답 없이 구조 발견

쇼핑몰 고객

구매 횟수와 평균 결제액만 주고 정답 그룹을 제공하지 않습니다. 모델이 "자주 소액 구매", "가끔 고액 구매", "최근 활동 없음" 같은 비슷한 고객 묶음을 발견합니다.

대표 작업

- **군집화:** 비슷한 데이터끼리 묶기
- **차원 축소:** 많은 특성을 핵심 정보로 압축하여 시각화·전처리
- **이상 탐지:** 일반 패턴에서 멀리 떨어진 사례 찾기

해석

군집 번호 0,1,2는 모델이 만든 이름표입니다. 실제 고객 유형의 의미는 사람이 원본 특성을 보고 해석해야 합니다.

강화학습: 행동 결과의 보상으로 학습

요소	게임 예시
에이전트	게임을 하는 AI
환경	게임판
상태	현재 말의 위치
행동	왼쪽·오른쪽 이동
보상	승리 +10, 패배 -10

한 행동의 즉시 결과뿐 아니라 이후에 받을 누적 보상을 높이는 행동 전략을 배웁니다.

지도학습과 차이

매 순간의 정답 행동을 알려주는 것이 아니라 시도한 결과의 보상으로 더 나은 행동을 탐색합니다.

개발환경 준비

프로젝트마다 독립된 가상환경을 사용하면 라이브러리 버전 충돌을 줄일 수 있습니다.

```
# PowerShell: 프로젝트 폴더로 이동
cd C:\work

# 가상환경 생성과 활성화
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 필수 라이브러리 설치
python -m pip install --upgrade pip
pip install numpy pandas scikit-learn matplotlib seaborn jupyter joblib
```

라이브러리	주요 역할
NumPy	숫자 배열, 난수, 수치 계산
Pandas	CSV와 표 데이터 처리
Scikit-learn	전처리, 회귀·분류, 평가, 교차검증
Matplotlib-Seaborn	분포, 예측, 혼동 행렬 시각화
Jupyter	셀 단위 실습 문서
joblib	학습 모델과 Pipeline 저장

중요

pip 대신 python -m pip를 사용하면 현재 활성화된 파이썬 환경에 설치하는지 더 명확하게 확인할 수 있습니다.

권장 프로젝트 구조

```
my-ml-project/
├─ data/
│  ├─ raw/          # 원본, 직접 수정하지 않음
│  └─ processed/    # 정리된 데이터
├─ notebooks/       # 탐색과 실험
├─ src/             # 재사용할 파이썬 코드
├─ models/          # 저장된 모델
├─ reports/         # 그래프와 보고서
├─ requirements.txt  # 라이브러리 버전
└─ README.md        # 실행 방법과 목표
```

파일 관리 원칙

- 원본 데이터는 보존하고 정리 결과를 별도 파일로 만듭니다.
- 절대경로를 코드 곳곳에 쓰기보다 프로젝트 기준 경로를 정합니다.
- 데이터 파일명, 타겟 열, 단위를 README에 기록합니다.
- 비밀번호·개인정보·API 키는 노트북이나 저장소에 넣지 않습니다.

현재 C:Wwork 프로젝트

dataset에 CSV, supervisedLearning에 노트북과 교재가 있습니다. 현재 구조를 유지하되 노트북의 ./dataset와 ../dataset 경로는 실행 위치에 맞춰 통일해야 합니다.

모델보다 먼저 문제를 정의한다

좋은 모델보다 올바른 질문이 먼저입니다.

질문	결정할 내용	예시
무엇을 예측하는가?	타겟 y	내일 매출
누구의 어떤 시점을 예측하는가?	데이터 한 행의 의미	매장별 하루
언제 예측하는가?	예측 시점에 사용 가능한 X	전날 밤
어떤 실수가 위험한가?	평가 지표	매출을 크게 낮게 예측
결과를 어떻게 사용할가?	의사결정	재고와 근무 인원 결정

미래 정보 누수

내일 매출을 예측하면서 내일 실제 방문객 수처럼 예측 시점에 알 수 없는 정보를 X 로 사용하면 현실에서는 사용할 수 없는 모델이 됩니다.

문제 정의 문장

“전날까지 알 수 있는 기온·예보·요일·과거 매출을 사용해 매장별 다음 날 매출을 예측하고, RMSE와 MAE로 평가한다.”처럼 작성합니다.

회귀인가, 분류인가?

타겟 y	문제	대표 지표
가격, 점수, 시간, 수량	회귀	MAE, RMSE, R ²
합격/불합격, 스팸/정상	이진 분류	정확도, 정밀도, 재현율, F1
붓꽃 3종, 상품 카테고리	다중 분류	혼동 행렬, macro F1

같은 학생 데이터
“몇 점인가?”는 회귀, “합격인가?”는 분류입니다. X가 아니라 y의 의미로 문제 유형을 결정합니다.

성공 기준도 함께 정한다

- 집값 MAE가 현재 수작업 예측보다 작아야 함
- 암 선별 모델은 정확도보다 환자를 놓치지 않는 재현율을 우선
- 스팸 필터는 정상 메일을 보호하기 위해 스팸 정밀도를 우선

CSV를 불러오면 가장 먼저 볼 것

```
from pathlib import Path
import pandas as pd

path = Path("data/raw/sales.csv")
df = pd.read_csv(path)

print(df.shape)      # 행과 열 개수
print(df.head())     # 처음 5행
print(df.info())     # 열 이름, 타입, 결측치
print(df.describe()) # 숫자 열의 범위와 분포
print(df.nunique())  # 열별 고유값 개수
```

확인	질문
행	한 행은 고객인가, 거래인가, 하루인가?
열	예측 시점에 실제로 사용할 수 있는 정보인가?
타입	숫자가 문자로 읽히지 않았는가?
범위	나이가 -3, 점수가 900처럼 불가능한 값이 있는가?
고유값	ID처럼 거의 모든 행이 다른 열인가?

원칙

모델을 실행하기 전에 데이터 한 행과 각 열의 의미를 말로 설명할 수 있어야 합니다.

X와 y 분리, 사용하면 안 되는 열

```
target = "sales"
X = df.drop(columns=[target])
y = df[target]

print(X.shape)
print(y.shape)
```

제외를 검토할 열

- **단순 ID:** 고객번호·거래번호는 관계가 없는데도 모델이 외울 수 있습니다.
- **타겟의 다른 표현:** 총매출을 예측하면서 세금 포함 총액을 X로 쓰면 답을 미리 제공하는 셈입니다.
- **예측 이후 생성되는 정보:** 연체 여부를 예측하면서 연체 후 추심 결과를 사용하면 안 됩니다.
- **개인정보:** 목적에 꼭 필요하고 적법한지 검토합니다.

상관관계가 높다고 무조건 사용하지 않기

타겟과 매우 높은 관계가 보이면 좋은 특성일 수도 있지만, 타겟에서 파생되었거나 미래 정보가 섞인 누수일 수도 있습니다.

결측치, 중복, 이상치

```
print(df.isna().sum())
print("duplicates:", df.duplicated().sum())
```

```
# 예: 가능한 범위 확인
print(df["age"].min(), df["age"].max())
```

문제	가능한 처리	주의
결측치	행 제거, 중앙값·최빈값 대체	결측 자체가 중요한 신호일 수 있음
중복	진짜 중복이면 제거	같은 고객의 정상적인 반복 거래일 수 있음
이상치	오류 수정, 제외, 별도 분석	실제 희귀 사례를 무조건 삭제하지 않음

학습 데이터에서만 기준 계산

중앙값으로 결측치를 채울 때도 학습 데이터의 중앙값을 구하고 시험 데이터에는 같은 값을 적용해야 합니다.

문자 범주를 숫자로 바꾸기

지역·요일·상품 종류는 단순히 1,2,3으로 바꾸면 잘못된 크기 관계가 생길 수 있습니다.

지역	강남	마포	송파
강남	1	0	0
마포	0	1	0
송파	0	0	1

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
```

`handle_unknown="ignore"`는 학습 때 없던 지역이 새 데이터에 등장해도 즉시 오류가 나지 않도록 합니다. 다만 새로운 범주가 자주 생긴다면 데이터 정책을 점검해야 합니다.

순서가 있는 범주

낮음·중간·높음처럼 실제 순서가 있는 값은 순서 정보를 반영할 수 있습니다. 지역처럼 순서가 없는 범주와 구분합니다.

전처리보다 먼저 데이터 분리

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

학습 데이터

결측치 기준, 표준화 기준, 어휘, 모델 계수를 학습합니다.

시험 데이터

모든 선택이 끝난 뒤 처음 보는 데이터처럼 최종 평가합니다.

분류에서는 클래스 비율 유지

```
train_test_split(
    X, y, test_size=0.2,
    random_state=42,
    stratify=y
)
```

시간 데이터

미래를 예측하는 데이터는 무작위로 섞지 않고 과거로 학습하고 이후 시점으로 시험해야 합니다.

Pipeline으로 누수 없는 전처리

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression

numeric = ["temperature", "visitors", "ad_cost"]
categorical = ["is_weekend", "region"]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

category_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric),
    ("cat", category_pipe, categorical)
])

model = Pipeline([
    ("preprocess", preprocess),
    ("model", LinearRegression())
])
```

Pipeline에 원본 DataFrame을 전달하면 학습 시 전처리 기준과 모델을 함께 학습하고, 예측 시 같은 처리를 자동 적용합니다.

복잡한 모델보다 기준 모델 먼저

새 모델이 실제로 유용한지 비교할 최소 기준이 필요합니다.

문제	간단한 기준
회귀	모든 사례에 학습 데이터 평균 또는 중앙값 예측
분류	가장 많은 클래스만 예측

```
from sklearn.dummy import DummyRegressor, DummyClassifier

baseline = DummyRegressor(strategy="mean")
# baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
```

판단

복잡한 모델이 기준 모델보다 좋아지지 않는다면 데이터, 특성, 문제 정의 또는 평가 방법부터 다시 확인합니다.

회귀의 큰 그림

회귀는 여러 사례에서 숫자의 변화 흐름을 찾아 새로운 숫자를 예측합니다.

공부 시간 x	실제 시험 점수 y
1시간	20점
2시간	30점
3시간	40점
4시간	50점

모델이 발견할 수 있는 흐름
공부 시간이 1시간 늘어날 때마다 점수가 약 10점 증가한다. 따라서 5시간 공부한 새로운 학생은 약 60점으로 예측한다.

x: 입력 조건
모델에게 알려주는 정보입니다. 공부 시간, 집 크기, 기온처럼 예측에 사용할 조건입니다.

y: 실제 정답
모델이 맞히려는 숫자입니다. 시험 점수, 집값, 매출처럼 최종 예측 대상입니다.



회귀인지 판단하는 가장 쉬운 질문

모델의 답이 “얼마인가?”라면 보통 회귀 문제입니다.

- 이 집의 가격은 얼마인가?
- 내일 기온은 몇 도인가?
- 배달은 몇 분 후 도착하는가?
- 시험 점수는 몇 점인가?

한 문장 핵심
회귀는 숫자가 입력되는 문제가 아니라, 최종적으로 숫자의 크기를 예측하는 문제입니다.

학습용과 시험용 데이터

모델이 관계를 이해했는지, 본 데이터를 외웠는지 구분하려면 데이터 역할을 나눠야 합니다.

학습용 train

답을 보며 공부하는 연습문제입니다. 모델이 x 와 y 의 관계를 찾을 때 사용합니다.

시험용 test

학습 때 보여주지 않은 시험문제입니다. 처음 보는 데이터에도 규칙이 통하는지 확인합니다.

구분	공부 시간	점수	사용 목적
학습용	1, 3, 5시간	20, 38, 59점	관계 학습
시험용	2, 4시간	32, 51점	새 데이터 예측 검사

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- **test_size=0.2**: 전체의 20%를 시험용으로 사용합니다.
- **random_state=42**: 매번 같은 방식으로 데이터가 섞이도록 고정합니다.

Random Seed

무작위를 없애는 설정이 아닙니다. 같은 무작위 결과를 다시 만들 수 있게 해 모델 변경 전후를 공정하게 비교합니다. 숫자 42 자체에는 특별한 의미가 없습니다.

왜 전부 학습에 쓰면 안 될까?

이미 정답을 본 연습문제를 다시 풀게 하고 실력이 좋다고 판단하는 것과 같습니다. 처음 보는 데이터에서의 성능을 알 수 없습니다.

선형회귀가 학습하는 두 가지

선형회귀는 데이터의 전체 흐름을 나타내는 직선을 찾습니다.

```
model = LinearRegression()
model.fit(X_train, y_train) # 공부
predictions = model.predict(X_test) # 시험
```

기울기

X가 1 증가할 때 예측 y가 얼마나 변하는지 나타냅니다.

0.79

"X가 1 늘면 y가 약 0.79 증가"

절편

X가 0일 때 모델이 예상하는 출발점입니다.

1.57

"X가 0이면 y는 약 1.57"

X=2의 예측

기본값 1.57에서 시작해 0.79를 두 번 더하면 약 3.15가 됩니다. 실제 모델 결과는 반올림 차이를 포함해 약 3.14입니다.

실제값, 예측값, 오차

X	실제값	예측값	빗나간 거리
2	4	3.14	약 0.86
5	5	5.50	0.50

실제 결과에는 수면, 기존 실력, 컨디션처럼 모델이 알지 못하는 조건도 영향을 줍니다. 그러므로 예측과 실제값의 차이는 자연스럽습니다.

한 문장 핵심

선형회귀의 목표는 모든 값을 외우는 것이 아니라, 여러 데이터의 전체적인 오차가 작아지는 직선 규칙을 찾는 것입니다.

MAE: 평균적으로 얼마나 빗나갔나

MAE는 예측이 정답에서 떨어진 거리를 모두 구한 뒤 평균을 냅니다.

실제값	예측값	빗나간 거리
3	2.5	0.5
-0.5	0	0.5
2	2	0
7	8	1

계산을 말로 읽기

거리 0.5, 0.5, 0, 1을 더하면 2입니다. 데이터 4개로 나누면 **MAE는 0.5**입니다.

왜 방향을 없앨까?

실제보다 10 낮은 예측과 10 높은 예측을 그대로 더하면 서로 지워져 0이 됩니다. MAE는 높고 낮은 방향 대신 **틀린 거리만** 보므로 평균적인 오차 크기를 알 수 있습니다.

시험 점수

MAE가 5라면 평균적으로 약 **5점**씩 빗나갔다는 뜻입니다.

집값

MAE가 1,000만원이라면 평균적으로 약 **1,000만원**씩 차이 난다는 뜻입니다.

한 문장 핵심

MAE는 모델이 정답에서 **평균적으로 얼마나 멀리** 빗나갔는지 원래 단위로 알려줍니다.

MSE와 RMSE: 큰 실수를 더 강하게 보기

MSE는 큰 오차에 더 큰 벌점을 주고, RMSE는 그 결과를 사람이 읽기 쉬운 원래 단위로 되돌립니다.

빗나간 거리	자기 자신과 한 번 더 곱한 값
1	1
2	4
3	9
10	100

MSE

큰 실수를 매우 크게 반영합니다. 하지만 단위도 두 번 곱해져 직접 해석하기 어렵습니다.

RMSE

MSE를 원래 단위로 되돌립니다. 큰 실수를 강조하면서 점, 원, 분처럼 읽을 수 있습니다.

점수와 집값에서 다르게 보이는 값

예측 대상	오차 사례	MSE	RMSE 해석
시험 점수	5, 5, 10, 10점	62.5점 ²	약 7.9점 규모로 빗나감
집값	1,000, 1,000, 2,000, 2,000만원	2,500,000만원 ²	약 1,581만원 규모로 빗나감

자주 하는 오해

MSE가 62.5라고 해서 평균 62.5점을 틀렸다는 뜻이 아닙니다. 사람이 원래 단위로 이해하려면 RMSE를 봅니다.

빠른 비교

MAE: 평균적인 거리 · **MSE**: 큰 실수에 강한 벌점 · **RMSE**: 큰 실수를 강조하면서 원래 단위로 표현

R²: 평균 예측보다 얼마나 나은가

R²는 오차의 단위가 아니라, 모델이 데이터의 흐름을 얼마나 잘 설명하는지 보여줍니다.

비교 기준

실제 점수가 50, 60, 70, 80점이면 평균은 65점입니다. 아무것도 학습하지 않은 방법은 모든 학생에게 65점을 예측할 수 있습니다.

R²	직관적인 의미
1에 가까움	평균으로 찍는 것보다 데이터 흐름을 매우 잘 설명
0에 가까움	평균값으로 예측하는 것과 비슷
0보다 작음	평균값으로 예측하는 것보다도 못함

학습 점수와 시험 점수를 함께 보기

```
model.score(X_train, y_train) # 학습 데이터 R²
model.score(X_test, y_test)   # 시험 데이터 R²
```

학습 R²	시험 R²	의심할 상태
낮음	낮음	관계를 충분히 배우지 못한 과소적합
매우 높음	낮음	학습 데이터를 외운 과적합
높음	높음	새 데이터에도 흐름이 잘 적용될 가능성

노트북 첫 예제의 R²가 음수인 이유

전체 데이터가 5개, 시험 데이터가 2개뿐이라 평가가 매우 불안정합니다. 약 -0.97이라는 결과가 코드 오류를 뜻하는 것은 아닙니다.

조건을 여러 개 사용하기

다중선형회귀는 여러 조건을 동시에 고려해 하나의 결과 숫자를 예측합니다.

공부 시간	과외 시간	시험 점수
2시간	0시간	60점
3시간	1시간	70점
5시간	2시간	85점
8시간	3시간	95점

```
X = np.column_stack((study_hours, tutor_hours))
y = test_scores
```

```
model.fit(X, y)
model.predict([[6, 2]]) # 공부 6시간, 과외 2시간
```

노트북 학습 결과

공부 시간 계수 약 4.87, 과외 시간 계수 약 1.94, 절편 약 53.39이며, 공부 6시간·과외 2시간의 예상 점수는 약 **86.5점**입니다.

계수는 이렇게 읽습니다

- 다른 조건이 같다면 공부 시간이 1시간 증가할 때 예측 점수가 약 4.87점 증가
- 다른 조건이 같다면 과외 시간이 1시간 증가할 때 예측 점수가 약 1.94점 증가

선에서 평면으로

조건이 하나면 회귀선, 조건이 두 개면 회귀 평면으로 시각화할 수 있습니다. 파란 점은 실제 학생, 빨간 평면은 모델의 예상이며 점과 평면 사이 거리가 오차입니다.

계수 해석 주의

계수가 크다고 반드시 더 중요한 원인인 것은 아닙니다. 변수마다 단위와 변화 범위가 다르고, 모델이 발견한 관계만으로 인과관계가 증명되지는 않습니다.

조건이 세 개여도 원리는 같다

노트북은 X1, X2, X3 세 조건으로 결과 Y를 만드는 가상 데이터를 사용합니다.

```
np.random.seed(42)
X1 = np.random.normal(0, 1, 100)
X2 = np.random.normal(0, 1, 100)
X3 = np.random.normal(0, 1, 100)

# Y는 X1, X2, X3의 영향과 약간의 무작위 오차를 포함
Y = 2 * X1 + 3 * X2 + 1.5 * X3 + noise
```

데이터를 만든 규칙은 모델에게 직접 알려주지 않습니다. 모델은 100개의 사례를 보고 각 조건의 영향력을 거꾸로 찾아냅니다.

조건	데이터를 만들 때 사용한 영향	모델이 학습한 계수
X1	약 2	약 1.79
X2	약 3	약 2.93
X3	약 1.5	약 1.51

시험 데이터 결과

R^2 는 약 **0.957**, RMSE는 약 **0.72**입니다. 무작위 오차가 섞여 있어 원래 계수와 완전히 같지는 않지만 전체 규칙을 상당히 가깝게 복원했습니다.

그래프로 그릴 수 없다고 학습할 수 없는 것은 아니다

조건 두 개까지는 3차원 평면으로 그릴 수 있지만 조건이 세 개 이상이면 사람이 한 장의 그래프로 보기 어렵습니다. 컴퓨터는 열이 더 많아져도 같은 방식으로 계수를 학습하고 예측할 수 있습니다.

한 문장 핵심

다중선형회귀는 조건의 수가 늘어나도 **다른 조건이 같을 때 각 조건이 결과와 어떻게 함께 변하는지** 학습합니다.

계수 해석에서 놓치기 쉬운 점

계수는 유용하지만 단위, 조건 사이의 중복 정보, 인과관계를 함께 고려해야 합니다.

1. 단위가 다르면 숫자 크기를 직접 비교하기 어렵다

집 크기는 평, 지하철 거리는 미터입니다. 집 크기 계수 1,000과 거리 계수 -2를 숫자만 비교하면 안 됩니다. 10평 차이와 1,000m 차이가 실제 예측에 주는 변화를 함께 봐야 합니다.

2. 비슷한 조건이 함께 있으면 계수가 불안정할 수 있다

집 크기와 방 개수처럼 거의 같은 정보를 가진 두 조건을 동시에 넣으면 모델은 어느 조건에 영향을 얼마나 나눠줄지 혼란스러울 수 있습니다. 데이터가 조금 바뀌었을 때 한 계수는 커지고 다른 계수는 작아질 수 있습니다. 이를 **다중공선성** 문제라고 합니다.

3. 계수는 원인 증명이 아니다

아이스크림 판매량과 물놀이 사고가 함께 증가하더라도 아이스크림이 사고를 일으킨다고 결론 내릴 수 없습니다. 더운 날씨라는 다른 조건이 둘을 함께 증가시킬 수 있습니다.

4. 표준화 후 계수는 비교가 조금 쉬워진다

조건들을 비슷한 숫자 범위로 맞추면 단위 차이의 영향을 줄일 수 있습니다. 그래도 계수가 인과관계나 절대적인 중요도를 증명하는 것은 아닙니다.

실무 해석

계수의 부호와 크기를 읽을 때는 단위, 조건의 실제 변화 범위, 서로 겹치는 정보, 빠진 조건을 함께 확인합니다.

CSV에서 X와 y 꺼내기

회귀 실습에서는 표에서 입력 열과 정답 열을 분리하는 작업이 반복됩니다.

```
dataset = pd.read_csv("../dataset/LinearRegressionData.csv")

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

표현	뜻
iloc	행과 열을 숫자 위치로 선택
:	모든 행
:-1	마지막 열을 제외한 앞의 모든 열
-1	마지막 열

열 이름을 알 때는 다음 코드가 더 읽기 쉽습니다.

```
X = dataset[["hour"]] # 표 형태 유지
y = dataset["score"] # 정답 목록
```

왜 X는 대괄호가 두 겹일까?

X는 조건의 표

```
[[0.5],
 [1.2],
 [1.8]]
```

각 행은 데이터 하나, 각 열은 조건 하나입니다.

y는 정답 목록

```
[10, 8, 14]
```

X의 각 행에 정답 하나가 대응합니다.

새 값 하나도 모델에는 표 형태로 전달합니다.

```
model.predict([[9]]) # 조건 1개인 데이터 1개
model.predict([[6, 2]]) # 조건 2개인 데이터 1개
```

한 문장 핵심

X는 여러 데이터와 여러 조건을 담은 표이고, y는 각 데이터에 대응하는 정답 목록입니다.

직선으로 부족할 때 곡선 사용하기

공부 시간이 늘어도 점수 상승 폭이 항상 같지는 않습니다. 처음에는 빠르게 오르고 나중에는 완만해질 수 있습니다.

공부 시간	점수	변화의 느낌
1시간	20점	출발
3시간	55점	빠르게 상승
5시간	84점	상승 폭 감소
7시간	94점	거의 완만

```
polyreg = make_pipeline(  
    PolynomialFeatures(degree=2),  
    LinearRegression()  
)  
polyreg.fit(X, y)
```

차수 degree	표현 능력
1	직선
2	완만하게 휘는 곡선
3 이상	더 복잡하게 휘는 곡선

노트북 결과
2차 다항회귀는 공부 3.8시간의 성적을 약 **50.7점**으로 예측하고, 전체 학습 데이터의 R²는 약 **0.976**입니다.

과소적합
모델이 너무 단순하여 학습 데이터와 시험 데이터 모두에서 흐름을 잘 잡지 못합니다.

과적합
곡선을 지나치게 복잡하게 만들어 학습 데이터의 우연한 흔들림까지 외웁니다.

좋은 모델
학습 데이터를 완벽히 통과하는 모델이 아니라, 처음 보는 시험 데이터에서도 전체 흐름을 잘 적용하는 모델입니다.

다항회귀 점수는 어떻게 검증할까?

노트북의 R^2 0.976은 모델을 학습시킨 바로 그 데이터에서 계산한 점수입니다.

```
polyreg.fit(X, y)
r_squared = polyreg.score(X, y) # 학습에 사용한 데이터로 평가
```

이미 답을 보며 곡선을 만든 뒤 같은 문제로 시험을 치른 결과이므로, 처음 보는 데이터에서도 0.976이 나온다고 보장할 수 없습니다.

더 공정한 평가

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

polyreg.fit(X_train, y_train)
print(polyreg.score(X_train, y_train))
print(polyreg.score(X_test, y_test))
```

결과	가능한 해석
학습·시험 점수 모두 낮음	곡선이 관계를 충분히 표현하지 못하는 과소적합 가능성
학습 점수만 매우 높음	학습 데이터의 흔들림까지 외운 과적합 가능성
둘 다 높고 차이가 작음	새 데이터에도 흐름이 적용될 가능성

차수는 시험 데이터로 계속 고르면 안 된다

degree 1, 2, 3을 시험 데이터에 반복 적용해 가장 좋은 것을 고르면 시험문제를 보며 모델을 선택하는 셈입니다. 차수 선택에는 학습 데이터 내부의 검증 또는 교차검증을 사용하고, 시험 데이터는 마지막 평가에 남겨둡니다.

한 문장 핵심

모델의 복잡도를 선택하는 점수와 최종 성능을 보고하는 시험 점수는 역할을 분리해야 합니다.

오차를 줄이는 방향으로 반복 수정

경사하강법은 적절한 기울기와 절편을 한 번에 맞히지 않고, 예측과 수정을 반복해 찾아갑니다.

예측

→

오차 확인

→

조금 수정

→

다시 예측

기울기 탐색 예시

실제 흐름이 “1시간마다 약 10점 증가”라면 모델은 $2 \rightarrow 5 \rightarrow 8 \rightarrow 9.5 \rightarrow 9.9$ 처럼 오차가 작아지는 방향으로 값을 수정할 수 있습니다.

산에서 낮은 곳 찾기

현재 위치에서 아래로 내려가는 방향을 확인하고 조금 움직인 뒤, 새로운 위치에서 다시 확인하는 과정을 반복한다고 생각하면 됩니다. 가장 낮은 곳은 예측 오차가 가장 작은 상태입니다.

```
sr = SGDRegressor(
    max_iter=200,
    eta0=1e-4,
    random_state=0
)
sr.fit(X_train, y_train)
```

설정	의미	주의
eta0	한 번에 수정하는 크기, 즉 학습률	너무 크면 지나치고, 너무 작으면 오래 걸림
max_iter	최대 반복 횟수	부족하면 안정적인 값에 도착하기 전에 종료
random_state	무작위 과정 재현	비교 실험에서 같은 값 사용

노트북의 수렴 경고

“Maximum number of iteration reached before convergence”는 200번 수정했지만 아직 충분히 안정적인 지점에 도착하지 못했다는 뜻입니다.

잔차: 오차를 방향까지 살펴보기

MAE와 RMSE는 오차를 하나의 숫자로 요약하지만, 잔차는 각 예측이 어느 방향으로 얼마나 틀렸는지 보여줍니다.

실제 점수	예측 점수	잔차	해석
80	70	+10	모델이 10점 낮게 예측
60	65	-5	모델이 5점 높게 예측
90	90	0	정확히 예측

좋은 잔차 그림

예측값이 작거나 크더라도 잔차가 0 위아래에 특별한 모양 없이 흩어져 있으면 모델이 놓친 뚜렷한 패턴이 적다는 신호입니다.

문제가 보이는 모양

- **활처럼 휨:** 실제 관계가 곡선인데 직선 모델을 사용했을 수 있습니다.
- **갈때기처럼 퍼짐:** 값이 커질수록 오차가 커져 예측의 안정성이 달라질 수 있습니다.
- **한쪽에 치우침:** 모델이 전반적으로 높게 또는 낮게 예측할 수 있습니다.
- **멀리 떨어진 점:** 이상치나 데이터 오류를 확인해야 합니다.

```
residuals = y_test - y_pred
plt.scatter(y_pred, residuals)
plt.axhline(0, color="red", linestyle="--")
```

한 문장 핵심

평가 점수 하나만 보지 말고 잔차에 남은 모양을 확인하면 모델이 놓친 관계를 발견할 수 있습니다.

선형회귀가 잘 맞으려면 확인할 것

아래 항목은 복잡한 공식을 외우기 위한 것이 아니라 결과를 지나치게 믿지 않기 위한 점검표입니다.

확인사항	쉬운 의미	확인 방법
관계의 모양	X가 변할 때 y의 평균 흐름이 대체로 직선인가?	산점도와 잔차 그림
오차의 독립성	한 예측의 오차가 다음 예측에 연달아 영향을 주지 않는가?	시간 순서 데이터인지 확인
오차 크기의 안정성	작은 값과 큰 값에서 오차의 퍼짐이 지나치게 달라지지 않는가?	잔차 그림의 깔때기 모양 확인
이상치	소수의 특이한 값이 회귀선을 끌어당기지 않는가?	산점도, 잔차, 원본 데이터 확인
조건의 중복	서로 거의 같은 정보를 가진 X들이 계수를 불안정하게 만들지 않는가?	특성 간 관계와 계수 변화 확인

예측과 통계 해석의 차이

오차가 완벽한 모양을 이루지 않아도 예측 모델로 사용할 수는 있습니다. 다만 계수의 통계적 의미나 불확실성을 엄밀하게 해석하려면 위 조건을 더 주의 깊게 확인해야 합니다.

시간 데이터

매출, 주가, 날씨처럼 시간 순서가 있는 데이터는 무작위 분할이 미래 정보를 과거 학습에 섞을 수 있습니다. 과거로 학습하고 더 나중 시점으로 시험하는 방식이 필요합니다.

결측치와 범주형 조건 처리

실제 데이터는 모든 값이 숫자로 완벽하게 채워져 있지 않습니다.

결측치

면적	층수	역과 거리	집값
24평	10층	500m	5억원
32평	값 없음	800m	6억원

값이 없으면 해당 행을 제거하거나, 학습 데이터에서 계산한 중앙값·평균 등으로 채울 수 있습니다. 시험 데이터의 값까지 함께 보고 채우는 기준을 만들면 데이터 누수가 됩니다.

범주형 조건

지역, 건물 유형, 요일처럼 글자로 된 조건은 모델이 사용할 숫자 열로 바뀌어야 합니다.

지역	강남 열	마포 열	송파 열
강남	1	0	0
마포	0	1	0
송파	0	0	1

지역을 강남=1, 마포=2, 송파=3으로만 바꾸면 모델이 송파가 강남보다 세 배 크다는 잘못된 숫자 관계를 학습할 수 있습니다. 그래서 각 종류를 별도의 0/1 열로 만드는 방식이 흔히 사용됩니다.

Pipeline으로 순서 묶기

결측치 처리, 표준화, 다항 특성 생성과 모델 학습을 Pipeline으로 묶으면 학습 데이터에서만 기준을 만들고 시험 데이터에 같은 처리를 적용하기 쉬워집니다.

한 문장 핵심

전처리도 모델 학습의 일부이므로 기준은 학습 데이터에서만 만들고 시험·새 데이터에는 그대로 적용합니다.

검증 데이터와 교차검증

모델 종류나 다항 차수, 학습률을 선택하면서 시험 데이터를 반복해서 보면 시험 데이터까지 외우게 됩니다.

학습 train

계수와 규칙을 학습

검증 validation

모델과 설정을 선택

시험 test

선택이 끝난 뒤 최종 평가

데이터가 적을 때 교차검증

학습 데이터를 여러 묶음으로 나누고, 번갈아 검증 역할을 맡깁니다. 한 번의 우연한 분할보다 여러 분할에서 안정적인 모델을 선택할 수 있습니다.

회차	학습 묶음	검증 묶음
1	A, B	C
2	A, C	B
3	B, C	A

데이터 누수의 대표 사례

- 전체 데이터로 표준화 기준을 만든 뒤 학습·시험을 분리
- 시험 데이터의 결측치까지 함께 보고 평균을 계산
- 시험 R^2 가 가장 높아지도록 degree를 반복 선택
- 미래 매출을 예측하면서 미래 정보가 포함된 열을 X로 사용

시험 데이터의 역할

시험 데이터는 학습 방법을 고치는 힌트가 아니라, 모든 선택이 끝난 모델이 처음 보는 상황에서 얼마나 작동하는지 확인하는 최종 시험입니다.

결과를 믿기 전에 확인할 것

1. 학습 범위 밖의 예측

1~10시간의 공부 데이터로 6시간을 예측하는 것은 비교적 자연스럽습니다. 하지만 100시간을 넣으면 직선을 그대로 연장해 1,000점 같은 비현실적인 결과가 나올 수 있습니다.

범위 안: 보간

이미 본 범위의 중간을 예측합니다. 주변 데이터가 있어 상대적으로 안정적입니다.

범위 밖: 외삽

본 적 없는 범위까지 규칙을 연장합니다. 현실의 제한을 몰라 위험할 수 있습니다.

2. 이상치

1~4시간의 점수가 20, 30, 40, 50점인데 5시간 학생만 5점이라면 마지막 값은 일반 흐름에서 크게 벗어납니다. 입력 실수, 특별한 사건, 실제 희귀 사례 중 무엇인지 먼저 확인해야 합니다.

3. 실제값·예측값 그래프

- 점이 대각선 가까이에 있을수록 실제값과 예측값이 비슷합니다.
- 대각선 위쪽이면 실제보다 높게, 아래쪽이면 낮게 예측했습니다.
- 선에서 멀리 떨어진 점은 크게 틀린 사례입니다.

4. 계수와 단위

집 크기는 평, 지하철 거리는 미터처럼 단위가 다릅니다. 계수 숫자만 비교해 "가장 중요한 특성"을 결정하면 잘못 해석할 수 있습니다. 필요하면 조건들의 기준을 맞추는 표준화를 사용합니다.

인과관계 주의

회귀가 "함께 변하는 관계"를 찾았다고 해서 한 조건이 결과를 직접 일으켰다는 사실까지 증명하는 것은 아닙니다.

조건이 많을 때 과적합 줄이기

조건이 매우 많거나 서로 비슷하면 계수가 지나치게 커지고 새 데이터에서 불안정해질 수 있습니다.

집값 예측에 면적, 방 수, 거실 크기, 창문 수, 건물 연식 등 많은 조건을 넣었더니 학습 데이터는 거의 완벽하지만 시험 데이터에서는 크게 틀릴 수 있습니다.

규제는 모델이 오차만 줄이려고 지나치게 큰 계수를 만드는 것을 억제해 더 단순하고 안정적인 규칙을 찾도록 돕습니다.

Ridge 회귀

계수들을 전반적으로 작게 눌러 여러 조건을 유지하면서 과도한 영향력을 줄입니다.

Lasso 회귀

일부 계수를 0까지 줄일 수 있어 사용하지 않는 조건을 선택하는 효과가 생길 수 있습니다.

```
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)

ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)
```

alpha가 커질수록 계수를 더 강하게 제한합니다. 너무 약하면 과적합을 줄이지 못하고, 너무 강하면 필요한 관계까지 약하게 만들어 과소적합될 수 있습니다. 적절한 값은 교차검증으로 선택합니다.

표준화

Ridge와 Lasso는 조건의 숫자 단위에 영향을 받으므로 보통 표준화와 함께 사용합니다.

범위 표시

Ridge와 Lasso는 원본 노트북에는 없지만, 선형회귀의 과적합과 계수 불안정을 이해하기 위한 필수 보강 개념입니다.

당뇨병 예제는 회귀가 아니라 분류

노트북은 diabetes.csv의 Outcome을 LinearRegression으로 예측하지만 Outcome은 0 또는 1의 클래스입니다.

값	의미	데이터 성격
0	당뇨병 음성	클래스 레이블
1	당뇨병 양성	클래스 레이블

LinearRegression을 사용하면 생기는 문제

- 예측값이 -0.1이나 1.2처럼 클래스 범위를 벗어날 수 있습니다.
- R^2 , MSE, 실제값-예측값 산점도는 이진 분류의 핵심 평가 방식을 보여주지 못합니다.
- 양성 환자를 놓친 경우와 정상인을 양성으로 잘못 경고한 경우를 구분하지 못합니다.

더 적절한 흐름

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, classification_report
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

정확도만 보지 말고 혼동 행렬, 양성 클래스의 정밀도·재현율·F1을 확인해야 합니다. 의료 데이터에서는 실제 환자를 놓치는 실수의 중요성도 고려해야 합니다.

특성 중요도 해석

원본 예제는 표준화하지 않은 선형회귀 계수의 절댓값을 중요도로 사용합니다. 혈당, BMI, 나이는 단위가 다르므로 계수 크기만으로 건강 지표 중요도를 비교하면 왜곡될 수 있습니다.

현재 회귀 노트북 실행 전 확인사항

항목	현재 상태	권장 조치
데이터 경로	dataset/..., ../dataset/..., ../dataset/... 혼용	os.getcwd() 확인 후 한 기준으로 통일
당뇨 Outcome	0/1 대상에 LinearRegression 사용	분류 모델과 분류 평가 지표 사용
다항회귀 평가	학습한 전체 데이터에서 R^2 계산	학습·시험 분리 또는 교차검증
SGDRegressor	200회 이전에 수렴하지 않았다는 경고 저장	표준화, 학습률, 반복 횟수 점검
셀 상태	X, y, model 이름을 여러 예제에서 반복 사용	새 커널에서 위에서부터 실행하고 예제별 변수명 사용
첫 단순회귀	시험 데이터가 2개뿐	R^2 를 일반 성능으로 해석하지 않기

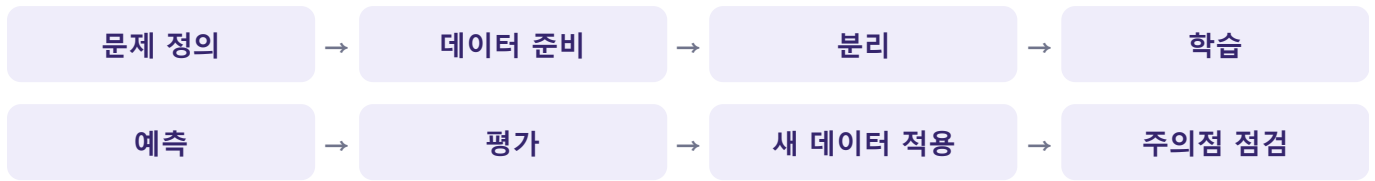
경로가 실행 위치에 따라 달라지는 이유

- 작업 폴더가 C:\work이면 dataset/file.csv가 맞습니다.
- 작업 폴더가 C:\work\supervisedLearning이면 ../dataset/file.csv가 맞습니다.
- 현재 프로젝트에는 회귀용 CSV 세 개가 C:\work\dataset에 존재합니다.

셀 실행 순서

Jupyter는 이전 셀의 변수를 기억합니다. 중간 셀만 실행하면 화면상 가까운 코드가 아니라 오래전에 만든 X_train이나 model을 사용할 수 있습니다. 커널을 재시작하고 위에서부터 실행하는 검사가 중요합니다.

회귀 전체 흐름 한 장 복습



1. **y 정하기**: 집값, 점수, 매출처럼 예측할 숫자를 정합니다.
2. **x 정하기**: 면적, 공부 시간, 방문객 수처럼 예측에 사용할 조건을 정합니다.
3. **데이터 분리**: 학습용으로 관계를 배우고 시험용으로 새 데이터 성능을 검사합니다.
4. **모델 학습**: `fit()`으로 입력 조건과 결과 사이의 흐름을 찾습니다.
5. **예측**: `predict()`로 시험 데이터와 새로운 데이터의 숫자를 예상합니다.
6. **평가**: MAE·RMSE로 오차 크기, R^2 로 평균 예측 대비 흐름 설명력을 봅니다.
7. **검토**: 과적합, 이상치, 단위, 학습 범위 밖 입력을 확인합니다.

최종 체크리스트

- ✓ x와 y의 역할을 설명할 수 있다.
- ✓ 학습용과 시험용 데이터를 나누는 이유를 설명할 수 있다.
- ✓ 기울기, 절편, 오차를 일상적인 말로 설명할 수 있다.
- ✓ MAE, MSE, RMSE, R^2 의 차이를 구분할 수 있다.
- ✓ 단순·다중·다항회귀의 차이를 예시로 설명할 수 있다.
- ✓ 경사하강법을 “예측과 수정의 반복”으로 설명할 수 있다.
- ✓ 과적합, 이상치, 외삽이 왜 위험한지 설명할 수 있다.
- ✓ 잔차 그림에서 곡선·깎때기·이상치를 찾을 수 있다.
- ✓ 전처리 기준을 학습 데이터에서만 만들어야 하는 이유를 안다.
- ✓ 검증 데이터와 시험 데이터의 역할을 구분할 수 있다.
- ✓ 0/1 레이블 대상은 일반적으로 회귀가 아니라 분류 문제임을 안다.

최종 한 문장

회귀는 과거의 입력 조건 **x**와 실제 숫자 **y**를 학습하여, 새로운 조건의 결과 숫자를 예측하는 지도학습 방법입니다.

노트북 코드 빠른 참조

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. 데이터 불러오기
dataset = pd.read_csv("../dataset/LinearRegressionData.csv")

# 2. 입력과 정답 분리
X = dataset[["hour"]]
y = dataset["score"]

# 3. 학습용과 시험용 분리
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# 4. 모델 생성과 학습
model = LinearRegression()
model.fit(X_train, y_train)

# 5. 시험 데이터 예측
y_pred = model.predict(X_test)

# 6. 평가
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred) ** 0.5)
print("R²:", r2_score(y_test, y_pred))

# 7. 새로운 데이터 예측
print("9시간 예상 점수:", model.predict([[9]])[0])
```

코드를 읽는 순서

read_csv로 가져오기 → X와 y 나누기 → train_test_split으로 역할 나누기 → fit으로 공부하기 → predict로 답하기 → 지표로 검사하기

노트북에서 기억할 실제 결과

예제	결과
공부 시간 전체 데이터 선형회귀	9시간 예상 약 93.8점, R^2 약 0.953
공부·과외 시간 다중회귀	6시간·2시간 예상 약 86.5점, 학습 R^2 약 0.961
X1·X2·X3 가상 다중회귀	시험 R^2 약 0.957, RMSE 약 0.72
2차 다항회귀	3.8시간 예상 약 50.7점, 학습 데이터 R^2 약 0.976

이진 분류와 다중 분류

이진 분류

두 종류 중 하나를 선택합니다.

- 합격 / 불합격
- 스팸 / 정상
- 질병 있음 / 없음
- 사과 / 사과 아님

다중 분류

세 종류 이상 중 하나를 선택합니다.

- Setosa
- Versicolor
- Virginica

붓꽃 데이터

꽃받침 길이	꽃받침 너비	꽃잎 길이	꽃잎 너비	y
5.1	3.5	1.4	0.2	0: Setosa
6.0	2.9	4.5	1.5	1: Versicolor
6.5	3.0	5.8	2.2	2: Virginica

레이블 2가 레이블 0보다 우수하거나 두 배라는 의미는 없습니다. 클래스 이름을 숫자로 치환했을 뿐입니다.

사진 분류에서 X와 y

- **x**: 실제 사진에서는 픽셀의 색상 숫자 배열. 사람이 미리 만든 시스템이라면 빨간색, 둥근 정도 같은 특징일 수도 있습니다.
- **y**: 이진 분류라면 사과 1 / 사과 아님 0, 다중 분류라면 사과·배·바나나 중 하나입니다.

판단 기준

분류의 종류는 입력 x가 아니라 **최종적으로 선택해야 하는 레이블의 개수**로 구분합니다.

가능성과 최종 레이블은 다르다

많은 분류 모델은 최종 선택 전에 각 클래스에 대한 점수나 가능성을 계산합니다.

단계	학생 합격 예시
실제 정답 y	합격 1
모델의 중간 판단	합격 가능성 0.80, 불합격 가능성 0.20
최종 예측	합격 1

임계값

이진 분류에서는 흔히 50%를 기본 기준으로 사용합니다.

합격 가능성	50% 기준의 최종 예측
20%	불합격
48%	불합격
65%	합격
90%	합격

임계값을 낮춤

더 많은 사례를 양성 1로 판단합니다. 실제 양성을 덜 놓치지만 잘못된 경고가 늘 수 있습니다.

임계값을 높임

더 확실한 사례만 양성 1로 판단합니다. 잘못된 경고는 줄지만 실제 양성을 놓칠 수 있습니다.

상황별 선택

암 검진은 환자를 놓치지 않기 위해 임계값을 낮출 수 있고, 스팸 필터는 정상 업무 메일을 보호하기 위해 임계값을 높일 수 있습니다.

중요한 보정

모든 분류 모델이 확률을 직접 계산하는 것은 아닙니다. 기본 SVM은 경계와의 거리를 사용합니다. 분류의 본질은 확률 그 자체가 아니라 **최종 클래스를 선택하는 결정 규칙**입니다.

로지스틱 회귀는 분류 모델

이름에는 회귀가 있지만, 입력이 특정 클래스에 속할 가능성을 구해 레이블을 선택합니다.

왜 선형회귀를 그대로 쓰지 않을까?

선형회귀로 합격 여부를 예측하면 0시간에 -0.2, 10시간에 1.5처럼 확률로 해석할 수 없는 값이 나올 수 있습니다. 확률은 0%와 100% 사이여야 합니다.

로지스틱 회귀는 내부 점수를 S자 형태로 눌러 0과 1 사이의 값으로 만듭니다.

공부 시간	합격 가능성 예시	최종 판단
1시간	5%	불합격
3시간	25%	불합격
5시간	75%	합격
7시간	95%	합격

학습과 예측 코드

```
classifier = LogisticRegression()
classifier.fit(X_train, y_train)

classifier.classes_      # 모델이 아는 클래스 [0, 1]
classifier.predict([[6]]) # 최종 레이블
classifier.predict_proba([[6]]) # 클래스별 가능성
```

predict_proba 읽기

classes_가 [0, 1]이고 결과가 [[0.15, 0.85]]이면 불합격 0은 15%, 합격 1은 85%입니다. 최종 예측은 합격 1입니다.

결정 경계

S자 곡선이 50% 임계값과 만나는 공부 시간이 불합격과 합격 판단이 바뀌는 지점입니다.

reshape: 값은 그대로, 모양만 변경

모델과 그래프 함수가 요구하는 행·열 모양에 맞추기 위해 배열을 재배치합니다.

```
p = np.array([[1, 2, 3], [4, 5, 6]])
# shape: (2, 3)
```

코드	결과 모양	직관적인 의미
<code>p.reshape(-1)</code>	(6,)	[1, 2, 3, 4, 5, 6] 한 줄 목록
<code>p.reshape(-1, 2)</code>	(3, 2)	열 2개, 행 수는 자동 계산
<code>p.reshape(3, -1)</code>	(3, 2)	행 3개, 열 수는 자동 계산
<code>X.reshape(-1, 1)</code>	(데이터 수, 1)	특성 한 개를 가진 입력 표

-1은 음수 크기가 아니라 전체 값의 수를 보고 해당 위치의 크기를 자동 계산하라는 뜻입니다.

가능성 그래프에서 사용

```
X_range = np.arange(min(X), max(X), 0.1)
p = p.reshape(-1)

plt.plot(X_range, p, color="green")
plt.plot(X_range, np.full(len(X_range), 0.5), color="red")
```

- 초록 곡선: 공부 시간별 합격 가능성
- 빨간 가로선: 50% 임계값
- 파란 점: 실제 합격·불합격 데이터

노트북 수정 필요

셀 50의 주석은 `reshape(3, -1)`이지만 실제 코드는 `reshape(-1, 2)`를 반복합니다. 결과 모양은 이 예제에서 같지만, 의도에 맞추려면 코드를 `p.reshape(3, -1)`로 바꿔야 합니다.

정확도와 불균형 데이터

정확도는 전체 예측 중 맞힌 비율이지만, 이것만으로 모델을 평가하면 위험할 수 있습니다.

전체 사례	맞힌 사례	정확도
학생 10명	8명	80%, 즉 0.8

```
accuracy_score(y_test, y_pred)
classifier.score(X_test, y_test) # 로지스틱 회귀의 기본 score
```

정확도 99%인데 쓸모없는 모델

1,000명 중 정상 990명, 환자 10명인 데이터에서 모든 사람을 정상으로 예측합니다.

실제	모델 예측	결과
정상 990명	모두 정상	990명 맞음
환자 10명	모두 정상	10명 모두 놓침

정확도는 99%지만 환자를 한 명도 찾지 못합니다. 한 클래스가 지나치게 많은 상태를 **클래스 불균형**이라고 합니다.

stratify

`train_test_split(..., stratify=y)`는 원본의 클래스 비율을 학습·시험 데이터에도 비슷하게 유지합니다. 특히 클래스 수가 적거나 불균형할 때 유용합니다. 단, 불균형 자체를 해결하는 기능은 아닙니다.

한 문장 핵심

정확도는 출발점일 뿐입니다. 어떤 클래스를 어떤 방향으로 틀렸는지 혼동 행렬과 클래스별 지표로 확인해야 합니다.

혼동 행렬: 어떤 방식으로 틀렸는가

질병 있음 1을 양성, 질병 없음 0을 음성으로 두고 네 가지 결과를 구분합니다.

실제 상태	정상으로 예측	환자로 예측
실제 정상	TN 80명 정확한 음성	FP 5명 잘못된 경고
실제 환자	FN 3명 환자를 놓침	TP 12명 환자를 발견

이름 읽는 법

- **True:** 예측이 맞음, **False:** 예측이 틀림
- **Positive:** 양성 1이라고 예측, **Negative:** 음성 0이라고 예측
- **False Negative:** 음성이라고 예측했지만 틀렸으므로 실제로는 양성

다중 클래스 혼동 행렬

붓꽃처럼 클래스가 3개라면 행과 열이 각각 Setosa, Versicolor, Virginica인 3×3 표가 됩니다.

- 대각선 칸: 실제와 예측 클래스가 같으므로 정답
- 대각선 밖 칸: 어느 품종을 다른 품종으로 혼동했는지 표시

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
             xticklabels=target_names,
             yticklabels=target_names)
```

한 문장 핵심

혼동 행렬은 “몇 개 맞았나”를 넘어 무엇을 무엇으로 잘못 판단했는지 보여줍니다.

정밀도와 재현율

같은 혼동 행렬을 서로 다른 질문으로 읽습니다.

정밀도

환자라고 한 판단을 얼마나 믿을 수 있는가?

환자라고 예측한 17명 중 실제 환자가 12명이면 약 71%입니다.

정상인을 환자라고 잘못 경고하는 FP를 줄이는 데 관심이 있습니다.

재현율

실제 환자를 얼마나 많이 찾아냈는가?

실제 환자 15명 중 12명을 발견하면 80%입니다.

환자를 정상이라고 놓치는 FN을 줄이는 데 관심이 있습니다.

상황	특히 중요한 지표	이유
스팸 필터	정밀도	정상 업무 메일을 스팸으로 보내지 않도록 주의
암 선별 검사	재현율	실제 환자를 가능한 한 놓치지 않도록 주의

임계값과의 관계

- 임계값을 높이면 확실한 사례만 양성으로 판단하여 정밀도가 오르는 경향이 있지만 재현율은 낮아질 수 있습니다.
- 임계값을 낮추면 양성을 더 많이 찾아 재현율이 오르는 경향이 있지만 잘못된 경고가 늘 수 있습니다.

정답은 상황에 따라 다름

정밀도와 재현율 중 항상 하나가 더 중요하지는 않습니다. 잘못된 경고와 놓침 중 어떤 실수가 더 위험한지 먼저 정해야 합니다.

F1 점수와 분류 보고서

F1은 정밀도와 재현율이 모두 괜찮은지 하나의 점수로 요약합니다.

정밀도	재현율	F1의 상태
90%	90%	높음
90%	30%	낮은 쪽의 영향으로 낮아짐
40%	95%	낮은 쪽의 영향으로 낮아짐
70%	75%	비교적 균형적

```
print(classification_report(
    y_test, y_pred,
    target_names=iris.target_names
))
```

어떤	뜻
precision	해당 클래스라고 예측한 판단의 정확성
recall	실제 해당 클래스를 얼마나 찾아냈는지
f1-score	정밀도와 재현율의 균형
support	시험 데이터에 실제 해당 클래스가 몇 개 있었는지

평균 항목

- **macro avg:** 각 클래스를 같은 중요도로 보고 평균. 소수 클래스 성능도 같은 무게로 반영합니다.
- **weighted avg:** support가 큰 클래스에 더 큰 비중을 주어 평균. 다수 클래스 영향이 큼.

평가 순서

정확도 → 혼동 행렬 → 클래스별 precision-recall-F1 → macro/weighted 평균 순으로 보면 모델의 실수 구조를 놓치기 어렵습니다.

KNN: 가까운 이웃의 다수결

새 데이터 주변에 어떤 클래스가 많은지 보고 분류합니다.

과일	무게	크기
사과 A	180g	8cm
사과 B	170g	7.5cm
귤 A	90g	5cm

새 과일

175g, 7.8cm인 과일의 가까운 이웃 3개가 사과·사과·귤이면 사과 2표로 최종 분류합니다.

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

K의 크기	특징	위험
너무 작음	아주 가까운 데이터에 민감	이상치 하나에 결과가 흔들릴 수 있음
적절함	주변의 지역적 패턴 반영	검증을 통해 선택 필요
너무 큼	멀리 있는 데이터까지 참여	다수 클래스의 영향이 지나치게 커질 수 있음

학습과 예측의 특징

KNN은 복잡한 규칙을 미리 만들기보다 학습 데이터를 기억합니다. 예측 시 새 데이터와 기존 데이터의 거리를 계산하므로 학습은 단순하지만 데이터가 많을수록 예측 비용이 커질 수 있습니다.

한 문장 핵심

KNN은 새로운 데이터와 가장 가까운 K개의 기존 사례를 찾고 **이웃들의 다수결**로 클래스를 결정합니다.

KNN에서 표준화가 중요한 이유

KNN은 거리 기반이므로 숫자 단위가 큰 조건이 거리를 지배할 수 있습니다.

무게 차이 50g과 크기 차이 2cm를 숫자 50과 2로 비교하면 무게가 훨씬 큰 영향을 줍니다. 무게가 실제로 더 중요해서가 아니라 단위가 크기 때문일 수 있습니다.

```
scaler = StandardScaler()

# 학습 데이터에서만 기준을 학습
X_train = scaler.fit_transform(X_train)

# 같은 기준을 시험·새 데이터에 적용
X_test = scaler.transform(X_test)
new_samples = scaler.transform(new_samples)
```

왜 시험 데이터에 fit하면 안 될까?

시험 데이터의 평균과 분포를 미리 사용하면 시험문제의 정보를 학습 과정에 흘려보내는 셈입니다. 이를 **데이터 누수**라고 합니다.

먼저 분리

→

train으로 fit

→

train 변환

→

test 변환

붓꽃 KNN

```
iris = load_iris()
X, y = iris.data, iris.target

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
predictions = knn.predict(new_samples)
```

노트북의 두 KNN 예제

첫 KNN 예제는 원본 길이를 그대로 사용하고, 뒤의 KNN 예제는 StandardScaler를 사용합니다. 붓꽃 네 특성은 모두 cm 단위라 큰 문제는 없지만, 거리 모델의 일반 원칙을 배우려면 두 번째 흐름이 더 안전합니다.

의사결정나무: 질문을 따라 분류

조건 질문을 순서대로 던지고 답에 따라 가지를 이동해 최종 클래스에 도착합니다.

```

꽃잎 길이가 2.5cm 이하인가?
├─ 예      → Setosa
└─ 아니요 → 꽃잎 너비가 1.7cm 이하인가?
               ├─ 예      → Versicolor
               └─ 아니요 → Virginica
  
```

어떤 질문을 고를까?

모델은 질문 전에는 여러 품종이 섞여 있지만 질문 후 각 그룹에 한 품종이 더 많이 모이도록 하는 기준을 찾습니다. 즉, 클래스를 가장 깔끔하게 나누는 특성과 기준값을 선택합니다.

```

clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
  
```

설정	뜻	주의
max_depth	질문을 이어갈 최대 단계	너무 깊으면 학습 데이터를 세세하게 외움
min_samples_split	추가 질문으로 나누기 위한 최소 데이터 수	작을수록 복잡한 나무 가능
min_samples_leaf	마지막 잎에 남아야 할 최소 데이터 수	작을수록 희귀 사례까지 따로 외울 수 있음

장점

판단 경로를 시각화하기 쉽고 표준화가 필수는 아닙니다.

단점

깊어지면 과적합되기 쉽고 데이터가 조금 바뀌어도 구조가 달라질 수 있습니다.

한 문장 핵심

의사결정나무는 데이터를 가장 잘 나누는 질문을 반복해 클래스가 비교적 순수한 마지막 그룹을 만듭니다.

랜덤포레스트: 여러 나무의 다수결

한 나무의 불안정성을 줄이기 위해 서로 조금씩 다른 나무를 여러 개 학습합니다.

나무	예측
나무 1	사과
나무 2	사과
나무 3	배
나무 4	사과
나무 5	배

사과 3표, 배 2표이므로 최종 예측은 사과입니다.

나무를 다르게 만드는 방법

- 각 나무가 학습할 사례를 무작위로 다시 뽑습니다. 같은 사례가 중복될 수도 있습니다.
- 질문을 만들 때 고려할 특성 후보도 무작위로 일부 선택합니다.
- 각 나무의 실수가 서로 다르면 다수결에서 잘못된 판단이 상쇄될 수 있습니다.

```
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
```

의사결정나무	랜덤포레스트
나무 한 개	나무 여러 개
판단 경로 설명이 쉬움	전체 판단 설명은 더 어려움
데이터 변화에 민감	비교적 안정적
과적합되기 쉬움	과적합 위험을 줄일 수 있음

한 문장 핵심

랜덤포레스트는 서로 다른 관점으로 학습한 여러 나무의 의견을 모아 한 나무보다 안정적인 판단을 만듭니다.

GridSearchCV와 교차검증

모델이 스스로 배우는 값이 아니라 사람이 학습 전에 정하는 설정을 하이퍼파라미터라고 합니다.

```
param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [4, 6, 8],
    "min_samples_split": [2, 4],
    "min_samples_leaf": [1, 2]
}

grid_search = GridSearchCV(
    estimator=rf_model,
    param_grid=param_grid,
    cv=3,
    scoring="accuracy"
)
```

cv=3의 동작

회차	학습에 사용	검증에 사용
1회	A, B 묶음	C 묶음
2회	A, C 묶음	B 묶음
3회	B, C 묶음	A 묶음

각 조합을 세 번 평가하고 결과를 종합합니다. 하나의 우연한 분할에서만 잘 나오는 설정보다 여러 분할에서 안정적인 설정을 찾습니다.

```
grid_search.fit(X_train, y_train)
grid_search.best_params_      # 가장 좋은 설정
best_rf_model = grid_search.best_estimator_

# 최종 시험 데이터는 마지막에 한 번 평가
y_pred = best_rf_model.predict(X_test)
```

평가 지표 선택

노트북은 `scoring="accuracy"`를 사용합니다. 클래스가 불균형하거나 소수 클래스를 놓치면 위험한 문제에서는 F1이나 재현율 등 목적에 맞는 지표를 선택해야 합니다.

데이터 역할

하이퍼파라미터 선택은 학습 데이터 내부의 교차검증으로 하고, 시험 데이터는 선택이 끝난 모델의 최종 평가에만 사용합니다.

랜덤포레스트의 특성 중요도

여러 나무가 질문을 만들 때 각 특성이 분류에 얼마나 유용했는지 요약합니다.

특성	중요도 예시	읽는 방법
꽃잎 길이	0.42	분류 질문에서 많이 활용
꽃잎 너비	0.38	분류 질문에서 많이 활용
꽃받침 길이	0.15	상대적으로 덜 활용
꽃받침 너비	0.05	현재 모델에서 활용이 적음

```
feature_importances = best_rf_model.feature_importances_
feature_importance_df = pd.DataFrame({
    "feature": feature_names,
    "importance": feature_importances
}).sort_values("importance", ascending=False)
```

해석할 때 주의

- 중요도가 높다는 것은 현재 데이터와 모델에서 많이 활용했다는 뜻입니다.
- 중요도가 높아도 결과를 일으킨 직접적인 원인이라는 뜻은 아닙니다.
- 비슷한 정보를 가진 두 특성은 중요도가 나뉘어 나타날 수 있습니다.
- 다른 데이터나 설정으로 학습하면 중요도 순서가 달라질 수 있습니다.

회귀계수와 차이

선형 모델의 계수는 조건 변화에 따른 예측 변화 방향과 크기를 나타냅니다. 랜덤포레스트 중요도는 나무들이 해당 특성을 이용해 데이터를 얼마나 유용하게 나눴는지 나타냅니다.

텍스트 분류 전체 흐름

문장을 그대로 모델에 넣을 수 없으므로 숫자 벡터로 변환한 뒤 분류합니다.

메일 문장

→

TF-IDF

→

숫자 벡터

→

분류 모델

스팸 데이터 전처리

```
data = pd.read_csv("./dataset/spam.csv", encoding="ISO-8859-1")
data = data[["v1", "v2"]]
data.columns = ["label", "text"]

data["label"] = data["label"].map({"ham": 0, "spam": 1})
data.dropna(inplace=True)
```

처리	의미
v1, v2 선택	레이블과 본문에 필요한 열만 사용
열 이름 변경	label, text로 역할을 명확하게 표현
ham/spam 매핑	정상 0, 스팸 1의 숫자 레이블로 변경
dropna	레이블이나 본문이 비어 있는 행 제거

encoding

파일이 UTF-8이 아니라 서유럽 문자 인코딩으로 저장돼 있을 수 있어 ISO-8859-1을 지정합니다. 잘못된 인코딩을 쓰면 문자가 깨지거나 읽기에 실패합니다.

클래스 비율

스팸 데이터는 정상 메일이 더 많은 경우가 많습니다. 분할 시 `stratify=data["label"]`을 추가하고 정확도 외에 스팸 클래스의 정밀도·재현율을 확인하는 편이 안전합니다.

TF-IDF: 중요한 단어를 숫자로 표현

특정 문서에서 자주 나오면서 다른 문서에서는 흔하지 않은 단어에 더 큰 값을 줍니다.

메일	무료	당첨	회의	일정
무료 상품 당첨	1	1	0	0
무료 쿠폰 제공	1	0	0	0
회의 일정 확인	0	0	1	1

단순 횟수에서 더 나아가 'the', 'is', 'and'처럼 모든 문서에서 흔한 단어의 중요도는 낮추고 '당첨', '무료', '회의'처럼 구분에 도움이 되는 단어를 강조합니다.

```
vectorizer = TfidfVectorizer(stop_words="english")

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

sample_tfidf = vectorizer.transform(sample_text)
```

- fit: 학습 메일에서 어휘와 중요도 기준을 학습
- transform: 문장을 같은 기준의 숫자 벡터로 변환
- stop_words="english": 분류에 도움이 적은 흔한 영어 단어 제거

데이터 누수 방지

시험 메일까지 fit_transform하면 시험 데이터의 단어 정보가 학습 기준에 섞입니다. 시험과 새 문장은 반드시 학습 데이터로 만든 vectorizer의 transform만 사용합니다.

나이브 베이즈: 단어를 증거로 판단

각 단어가 스팸과 정상 메일에서 얼마나 자주 등장했는지 학습합니다.

스팸에서 흔한 단어

무료, 당첨, 클릭, 상금, 축하합니다

정상에서 흔한 단어

회의, 일정, 보고서, 프로젝트, 첨부

새 메일

“축하합니다. 무료 상품에 당첨되었습니다. 지금 링크를 클릭하세요.”에는 스팸에서 흔한 증거가 많이 포함되어 스팸으로 분류될 가능성이 높습니다.

왜 Naive, 즉 순진한가?

현실에서는 무료·상품·당첨이 서로 관련될 수 있지만, 계산을 단순하게 하기 위해 각 단어를 독립적인 증거처럼 다룹니다. 완벽한 가정은 아니지만 조건 수가 많은 텍스트에서 빠르고 효과적일 수 있습니다.

```
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

y_pred = model.predict(X_test_tfidf)
predictions = model.predict(sample_tfidf)
```

장점	주의점
학습·예측이 빠르고 텍스트 빈도 정보에 잘 맞춤	단어들이 독립적이라는 단순한 가정을 사용
조건이 매우 많은 희소 벡터 처리에 적합	문맥과 단어 순서를 직접 이해하는 모델은 아님

한 문장 핵심

나이브 베이즈는 새 문장의 단어들이 어느 클래스에서 더 자연스럽게 등장할 조합인지 판단합니다.

SVM: 가장 여유 있는 경계 찾기

두 클래스 사이에 여러 경계가 가능할 때 양쪽 데이터에서 최대한 멀리 떨어진 경계를 선택합니다.

과일 예시

사과 그룹과 귤 그룹 사이의 가운데에 넓은 여유 공간을 두는 경계를 찾으면 새 데이터가 조금 흔들려도 분류가 쉽게 바뀌지 않습니다.

마진과 Support Vector

- **마진:** 경계와 양쪽 클래스 사이의 여유 공간
- **Support Vector:** 경계에 가장 가까워 경계 위치를 결정하는 핵심 데이터

```
model = SVC(kernel="linear", random_state=42)
model.fit(X_train_tfidf, y_train)
y_pred = model.predict(X_test_tfidf)
```

kernel="linear"는 직선·평면·더 높은 차원의 평평한 경계를 사용한다는 뜻입니다. TF-IDF처럼 특성이 매우 많은 텍스트 데이터에서도 사용할 수 있습니다.

장점	주의점
클래스 사이의 안정적인 경계를 찾음	일반 숫자 데이터에서는 표준화가 중요
특성이 많은 텍스트 데이터에 활용 가능	데이터가 매우 크면 학습이 느릴 수 있음
경계 근처 핵심 데이터에 집중	기본 SVC는 확률을 직접 제공하지 않음

커널의 역할

선형 경계로 나누기 어려운 데이터에는 곡선 경계를 만들 수 있는 다른 커널도 있습니다. 다만 노트북은 텍스트 분류에 linear를 사용합니다.

분류 모델 한눈에 비교

모델	핵심 판단 방식	표준화	설명 용이성
로지스틱 회귀	가능성과 임계값	권장	비교적 쉬움
KNN	가까운 이웃 다수결	중요	쉬움
의사결정나무	조건 질문	필수 아님	매우 쉬움
랜덤포레스트	여러 나무 다수결	필수 아님	보통
나이브 베이즈	클래스별 특징 출현	상황에 따라 다름	보통
SVM	가장 여유 있는 경계	중요	어려운 편

상황별 출발점

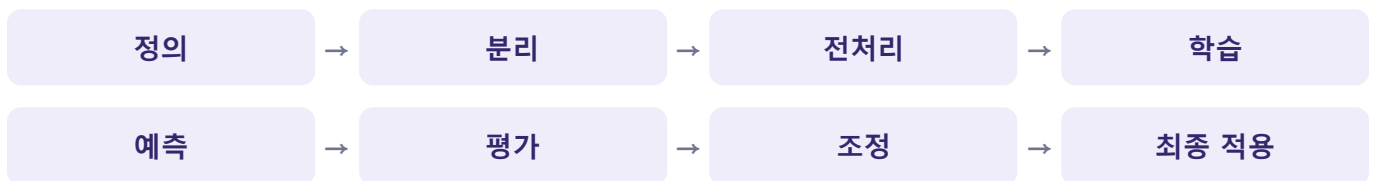
- 가능성과 조건 방향을 보고 싶다: 로지스틱 회귀
- 비슷한 사례끼리 모이고 데이터가 적다: KNN
- 판단 근거를 질문 형태로 설명해야 한다: 의사결정나무
- 표 데이터에서 안정적인 기본 성능이 필요하다: 랜덤포레스트
- 빠른 텍스트 분류 기준 모델이 필요하다: 나이브 베이즈
- 특성이 많은 데이터에서 분명한 경계를 찾고 싶다: SVM

최고의 모델은 미리 정해지지 않음

데이터 크기, 클래스 불균형, 설명 가능성, 예측 속도, 어떤 실수가 더 위험한지에 따라 선택이 달라집니다. 같은 분할과 같은 평가 기준으로 여러 모델을 비교해야 합니다.

분류 프로젝트 전체 흐름

1. **문제 정의:** 어떤 레이블을 예측할지, 어떤 실수가 더 위험한지 정합니다.
2. **X와 y 준비:** 입력 특징과 실제 클래스 레이블을 분리합니다.
3. **데이터 확인:** 결측치, 클래스 개수와 비율, 이상치를 확인합니다.
4. **학습·시험 분리:** 필요하면 stratify=y로 클래스 비율을 유지합니다.
5. **전처리:** 표준화나 TF-IDF를 학습 데이터에서만 fit합니다.
6. **모델 학습:** fit(X_train, y_train)을 실행합니다.
7. **예측:** predict(X_test)로 시험 레이블을 구합니다.
8. **평가:** 정확도, 혼동 행렬, 정밀도, 재현율, F1을 확인합니다.
9. **설정 선택:** 필요하면 학습 데이터 내부의 교차검증을 사용합니다.
10. **새 데이터:** 학습 때 만든 동일한 전처리 기준을 적용한 뒤 예측합니다.



반드시 피할 데이터 누수

```

# 올바른 예
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 잘못된 예
scaler.fit(X_test) # 시험 정보를 미리 봄
  
```

한 문장 핵심

시험 데이터는 모델과 전처리 기준을 선택하는 데 사용하지 않고, 모든 선택이 끝난 뒤 처음 보는 문제처럼 평가합니다.

현재 노트북 실행 전 확인사항

개념과 별개로 현재 프로젝트 상태에서는 일부 셀이 그대로 실행되지 않습니다.

항목	현재 상태	필요한 조치
LogisticRegressionData.csv	C:\work\dataset에 없음	파일 추가 또는 올바른 경로·데이터로 수정
spam.csv	C:\work\dataset에 없음	파일 추가 후 v1, v2 열 존재 확인
데이터 경로	./dataset와 ../dataset 혼용	노트북 실행 위치를 기준으로 경로 통일
reshape 셀 50	주석과 코드가 다름	p.reshape(3, -1)로 의도 명확화
첫 KNN 예제	표준화 없음	일반 원칙 학습에는 StandardScaler 흐름 권장
분류 데이터 분할	일부 예제에 stratify 없음	클래스 비율 유지가 필요하다면 stratify=y 추가

경로가 달라지는 이유

- 작업 폴더가 C:\work이면 ./dataset/file.csv가 맞습니다.
- 작업 폴더가 C:\work\supervisedLearning이면 ../dataset/file.csv가 맞습니다.
- os.getcwd()로 현재 작업 폴더를 확인한 뒤 한 방식으로 통일해야 합니다.

중요

CSV 파일이 없어도 scikit-learn에 내장된 붓꽃 예제는 실행할 수 있습니다. 로지스틱 회귀와 스팸 예제는 필요한 외부 데이터 파일을 먼저 준비해야 합니다.

표 데이터 분류 코드 빠른 참조

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

# 1. 분리: 클래스 비율 유지
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 2. 전처리: train에서만 기준 학습
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. 모델 학습과 예측
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)

# 4. 평가
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

# 5. 새로운 데이터도 같은 기준으로 변환
new_scaled = scaler.transform(new_data)
print(model.predict(new_scaled))
print(model.predict_proba(new_scaled))
```

모델 교체

같은 전처리·평가 구조에서 LogisticRegression을 KNN, 의사결정나무, 랜덤포레스트, SVM 등으로 바꾸어 비교할 수 있습니다. 단, 모델마다 표준화 필요성과 제공 기능이 다릅니다.

최종 복습 체크리스트

- ✓ 회귀와 분류를 y 의 성격으로 구분할 수 있다.
- ✓ 이진 분류와 다중 분류의 차이를 설명할 수 있다.
- ✓ 실제 레이블, 클래스 가능성, 최종 예측을 구분할 수 있다.
- ✓ 임계값을 바꾸면 정밀도와 재현율이 달라지는 이유를 안다.
- ✓ 정확도만으로 불균형 데이터를 평가하면 위험한 이유를 안다.
- ✓ TP, TN, FP, FN을 질병 예시로 설명할 수 있다.
- ✓ 정밀도, 재현율, F1, support의 의미를 구분할 수 있다.
- ✓ KNN의 K와 표준화 필요성을 설명할 수 있다.
- ✓ 의사결정나무와 랜덤포레스트의 차이를 설명할 수 있다.
- ✓ GridSearchCV가 학습 데이터 내부에서 설정을 비교하는 이유를 안다.
- ✓ TF-IDF가 문장을 숫자로 바꾸는 이유를 설명할 수 있다.
- ✓ 나이브 베이즈와 SVM의 판단 방식을 구분할 수 있다.
- ✓ 전처리 기준을 시험 데이터에 fit하면 안 되는 이유를 안다.

모델별 한 문장

모델	기억할 문장
로지스틱 회귀	가능성을 구해 임계값으로 레이블 선택
KNN	가까운 이웃들의 다수결
의사결정나무	조건 질문을 따라 마지막 클래스 도착
랜덤포레스트	서로 다른 여러 나무의 다수결
나이브 베이즈	특징이 클래스별로 등장하는 경향을 증거로 사용
SVM	클래스 사이의 가장 여유 있는 경계 탐색

최종 한 문장

분류는 과거의 입력 x 와 정답 레이블 y 를 학습하여, 새로운 입력이 어느 클래스에 속하는지 결정하는 지도학습 방법입니다.

Loss와 평가 지표는 역할이 다르다

Loss 손실함수

학습 중 모델이 얼마나 틀렸는지 계산해 어느 방향으로 수정할지 알려줍니다.

Metric 성능지표

사람이 모델을 비교하고 업무 목적에 적합한지 판단합니다.

내비게이션 비유

Loss는 운전 중 목적지까지 거리를 계속 확인하는 계기판입니다. 최종 평가는 도착 시간, 연료비, 안전성처럼 목적에 맞는 여러 기준입니다.

상황	학습 Loss 예	보고 지표 예
집값 회귀	MSE	MAE, RMSE, R ²
이진 분류	Log Loss	정밀도, 재현율, F1

곡선 해석

학습 Loss는 계속 낮아지는데 검증 Loss가 다시 높아지면 학습 데이터를 외우는 과적합 신호일 수 있습니다.

앙상블: 여러 모델의 판단을 합치기

방식	학습 방식	대표 모델
Bagging	서로 다른 표본으로 모델을 독립적으로 학습하고 평균·다수결	Random Forest
Boosting	앞 모델이 틀린 사례를 다음 모델이 순차적으로 보완	XGBoost, LightGBM

시험 문제 비유

Bagging은 여러 학생이 독립적으로 답을 풀고 다수결합니다. Boosting은 첫 학생의 오답을 다음 학생이 집중 복습하며 이어서 개선합니다.

XGBoost와 LightGBM

- 표 형태 데이터에서 강력한 성능을 내는 경우가 많습니다.
- 학습률, 트리 깊이, 트리 수 등 설정 조정이 중요합니다.
- 높은 정확도가 항상 쉬운 해석이나 과적합 방지를 보장하지는 않습니다.

원 강의 표현 보정

Boosting은 앞선 오류를 순차적으로 보완합니다. “병렬 처리”는 구현의 일부 계산 최적화일 수 있지만 트리 학습 개념 자체는 순차적입니다.

원 강의자료를 읽을 때 교정할 부분

원 자료 표현	교정
KNN 분류 코드에 KNeighborsRegressor	분류라면 KNeighborsClassifier 사용
학습 객체 knn_regressor, 예측은 model	같은 변수명으로 predict 실행
다항회귀는 S자 곡선	차수와 계수에 따라 다양한 곡선이며 반드시 S자가 아님
다중회귀는 상호작용 자동 고려	곱 같은 상호작용 특성을 별도로 추가해야 함
다중회귀가 더 높은 예측력	불필요한 특성은 과적합을 만들 수 있어 검증 필요
회귀 출력 0.0~1.0	일반 회귀 출력 범위는 타겟에 따라 달라짐

학습 태도

슬라이드는 핵심을 빠르게 소개하는 자료입니다. 코드는 실행 결과와 데이터 의미를 확인하면서 교재의 보강 설명과 함께 읽습니다.

교차검증으로 안정성 확인

한 번의 학습·시험 분할 결과는 우연히 좋거나 나쁠 수 있습니다. 학습 데이터를 여러 묶음으로 나누어 번갈아 검증합니다.

```
from sklearn.model_selection import cross_validate

scores = cross_validate(
    model, X_train, y_train,
    cv=5,
    scoring=["neg_mean_absolute_error", "r2"]
)
```

- 평균 점수: 일반적인 성능 수준
- 분할별 차이: 데이터에 따라 성능이 얼마나 흔들리는지
- 시험 데이터: 모델·설정 선택이 끝난 뒤 마지막 한 번 평가

전처리 포함

교차검증에는 전처리와 모델이 묶인 Pipeline 전체를 전달해야 각 검증 묶음의 정보가 학습에 새지 않습니다.

하이퍼파라미터 조정

```
from sklearn.model_selection import GridSearchCV

params = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [None, 5, 10]
}

search = GridSearchCV(
    model, params,
    cv=5,
    scoring="neg_mean_absolute_error"
)
search.fit(X_train, y_train)
best_model = search.best_estimator_
```

지표는 목적에 맞게

회귀에서 큰 실수가 위험하면 RMSE, 평균적인 금액 차이가 중요하면 MAE를 사용합니다. 분류에서 환자 놓침이 위험하면 재현율, 균형이 중요하면 F1을 선택합니다.

많이 시험할수록 좋은 것은 아님

근거 없이 거대한 조합을 탐색하면 시간만 늘고 검증 데이터에 과적합될 수 있습니다. 먼저 중요한 설정과 현실적인 범위를 정합니다.

점수 다음에는 틀린 사례를 본다

회귀

- 오차가 가장 큰 행을 정렬해 공통점을 찾습니다.
- 특정 지역·가격대·시기에서 계속 낮게 예측하는지 확인합니다.
- 잔차가 곡선이나 깔때기 모양인지 봅니다.

분류

- FP와 FN 사례를 따로 확인합니다.
- 소수 클래스가 주로 틀리는지 봅니다.
- 업무상 더 위험한 실수를 줄이도록 임계값과 지표를 조정합니다.

```
# 회귀 오차가 큰 사례
result = X_test.copy()
result["actual"] = y_test
result["predicted"] = y_pred
result["abs_error"] = (result["actual"] - result["predicted"]).abs()
print(result.sort_values("abs_error", ascending=False).head(10))
```

개선의 출발점

평균 점수를 조금 높이는 것보다 실제로 어떤 데이터에서 왜 실패하는지 이해하는 것이 더 유용할 수 있습니다.

전처리와 모델을 함께 저장

```
import joblib

# Pipeline 전체 저장
joblib.dump(best_model, "models/sales_model.joblib")

# 다시 불러오기
loaded_model = joblib.load("models/sales_model.joblib")
prediction = loaded_model.predict(new_data)
```

스케일러나 OneHotEncoder를 따로 저장하지 않고 Pipeline 전체를 저장하면 새 데이터에도 학습 때와 같은 순서가 자동 적용됩니다.

함께 기록할 것

- 모델 파일 생성 날짜와 버전
- 사용한 데이터 기간과 타겟 정의
- 입력 열 이름, 순서, 타입, 단위
- 라이브러리 버전
- 시험 데이터 평가 결과
- 알려진 한계와 사용하면 안 되는 상황

보안

신뢰할 수 없는 출처의 joblib-pickle 모델 파일은 불러오지 않습니다. 로딩 과정에서 임의 코드가 실행될 수 있습니다.

새 데이터 예측 전 검증

```
required = ["temperature", "visitors", "ad_cost", "is_weekend", "region"]

missing = set(required) - set(new_data.columns)
if missing:
    raise ValueError(f"Missing columns: {sorted(missing)}")

prediction = loaded_model.predict(new_data[required])
```

확인사항

- 학습 때와 같은 열 이름과 의미인가?
- 원과 만원, m와 km처럼 단위가 바뀌지 않았는가?
- 숫자 열이 문자열로 들어오지 않았는가?
- 학습 범위를 크게 벗어난 값이 아닌가?
- 새로운 범주와 결측치를 Pipeline이 처리할 수 있는가?

예측값은 사실이 아님

모델 출력은 과거 데이터에서 학습한 추정치입니다. 중요한 의사결정에서는 입력 범위, 불확실성, 사람의 검토 절차를 함께 둡니다.

같은 결과를 다시 만드는 기록

```
# 주요 무작위 설정 고정
random_state = 42

# 설치 버전 기록
pip freeze > requirements.txt
```

기록 항목	예시
실험 이름	sales_rf_v3
데이터	2025-01~2025-12, 18,420행
분할	시간 기준, 마지막 2개월 시험
특성	기온, 요일, 과거 7일 매출
모델·설정	RandomForest, depth=10
결과	MAE 82만원, RMSE 121만원

Seed의 한계
Random Seed는 같은 무작위 순서를 재현하도록 돕지만 데이터·코드·라이브러리 버전이 달라지면 결과도 달라질 수 있습니다.

회귀 프로젝트 완성 템플릿

```
import joblib
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("data/raw/sales.csv")
X = df.drop(columns=["sales"])
y = df["sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

num = ["temperature", "visitors", "ad_cost"]
cat = ["is_weekend", "region"]

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), num),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), cat)
])

model = Pipeline([
    ("preprocess", preprocess),
    ("model", Ridge(alpha=1.0))
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("MAE", mean_absolute_error(y_test, y_pred))
print("RMSE", mean_squared_error(y_test, y_pred) ** 0.5)
print("R2", r2_score(y_test, y_pred))

joblib.dump(model, "models/sales_model.joblib")
```

자주 발생하는 문제

현상	주요 원인	확인
FileNotFoundError	작업 폴더와 상대경로 불일치	os.getcwd(), 파일 존재
could not convert string to float	문자 범주 미처리	OneHotEncoder 사용
Input contains NaN	결측치 남음	SimpleImputer 사용
열 개수·이름 오류	학습·예측 입력 구조 다름	Pipeline과 required 열 확인
학습은 좋은데 시험은 나쁨	과적합, 누수, 분포 차이	교차검증, 복잡도 축소, 오류 분석
정확도만 매우 높음	클래스 불균형	혼동 행렬, recall, F1
실행마다 결과가 다름	Seed·분할·버전 차이	random_state와 버전 기록
SGD 수렴 경고	스케일, 학습률, 반복 횟수	표준화와 max_iter 점검

디버깅 순서

오류 메시지 마지막 줄 → 현재 작업 폴더 → 데이터 shape·열 이름·dtype → 결측치 → 학습/예측 전처리
일치 순으로 확인합니다.

프로젝트 완료 체크리스트

- ✓ 예측 시점, 타겟 y , 데이터 한 행의 의미를 문장으로 정의했다.
- ✓ 회귀/분류를 y 의 성격으로 선택했다.
- ✓ 예측 시점에 알 수 없는 열과 ID-타겟 누수 열을 제외했다.
- ✓ 결측치, 중복, 이상치, 타입과 단위를 확인했다.
- ✓ 전처리 전에 학습·시험 데이터를 분리했다.
- ✓ 전처리와 모델을 Pipeline으로 묶었다.
- ✓ 간단한 기준 모델보다 나은지 확인했다.
- ✓ 목적에 맞는 평가 지표를 선택했다.
- ✓ 교차검증으로 결과의 흔들림을 확인했다.
- ✓ 가장 크게 틀린 실제 사례를 분석했다.
- ✓ 시험 데이터는 모든 선택 후 마지막 평가에 사용했다.
- ✓ Pipeline 전체와 입력 스키마를 함께 저장했다.
- ✓ 데이터 기간, 코드, 라이브러리 버전, Seed와 결과를 기록했다.
- ✓ 모델의 한계와 사용하면 안 되는 상황을 문서화했다.

최종 원칙

좋은 머신러닝 프로젝트는 복잡한 모델보다 **명확한 문제, 누수 없는 데이터 처리, 공정한 평가, 재현 가능한 기록**에서 시작합니다.

하고 싶은 일로 메서드 찾기

하고 싶은 일	도구	사용 단계
경로 만들기·존재 확인	Path(), exists()	파일 로드 전
CSV 읽기	pd.read_csv()	데이터 수집 후
구조 확인	head(), info(), describe()	모델 선택 전
결측·중복 확인	isna(), duplicated()	데이터 정리
배열 모양 변경	reshape(), column_stack()	모델 입력 준비
학습·시험 분리	train_test_split()	전처리 fit 전
숫자 기준 맞춤	StandardScaler	거리·선형 모델 전
문자 범주 변환	OneHotEncoder	모델 학습 전
전처리 연결	Pipeline, ColumnTransformer	학습 설계
학습·예측	fit(), predict()	모델 단계
설정 탐색	GridSearchCV	학습 데이터 내부
모델 저장	joblib.dump()	최종 평가 후

메서드를 설명하는 고정 형식

예: `pd.read_csv()`

역할: CSV를 DataFrame으로 읽습니다.

사용 단계: 파일 경로 확인 후 최초 로드.

입력: 경로와 옵션. **출력:** DataFrame.

```
df = pd.read_csv(
    "data.csv",
    encoding="utf-8",
    usecols=["age", "income", "target"],
    na_values=["?", "unknown"]
)

# 사용 직후 확인
print(df.shape)
print(df.head())
df.info()
```

항목	확인 내용
역할	무슨 작업을 수행하는가?
프로젝트 단계	분리 전·후, 학습 전·후 중 어디인가?
입력과 출력	경로, DataFrame, 배열, 모델 중 무엇인가?
주요 옵션	결과를 바꾸는 핵심 설정은 무엇인가?
자주 하는 실수	경로, shape, 누수, 타입 문제는 없는가?
다음 단계	이 결과를 어떤 도구에 전달하는가?

원칙

메서드 이름을 암기하기보다 입력이 무엇이고 출력이 다음 어느 단계로 전달되는지 연결합니다.

공통 메서드와 대상

메서드	대상	역할
fit(X,y)	모델	X와 y의 규칙 학습
fit(X)	전처리기	평균·어휘·범주 같은 기준 학습
transform(X)	전처리기	학습한 기준 적용
fit_transform(X)	전처리기	기준 학습과 적용을 연속 수행
predict(X)	모델·Pipeline	숫자 또는 클래스 예측
predict_proba(X)	지원 분류 모델	클래스별 가능성
score(X,y)	모델	기본 점수의 빠른 확인
get_params()	Estimator	설정 조회와 기록

누수 방지

시험 데이터에는 전처리기의 fit이나 fit_transform을 실행하지 않고, 학습 데이터에서 만든 기준으로 transform만 실행합니다.

Matplotlib 기본 구조와 한글 설정

```
import matplotlib.pyplot as plt

# 공통 비주얼 설정
plt.rcParams["font.family"] = "Malgun Gothic"
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 120

plt.figure(figsize=(10, 6))
plt.scatter(X, y, color="blue", label="실제 데이터")
plt.plot(X, y_pred, color="red", label="예측 결과")

plt.xlabel("공부 시간 (시간)")
plt.ylabel("성적")
plt.title("공부 시간에 따른 성적 예측 (다항 회귀)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

코드	비주얼 역할
figure()	그림판 크기와 해상도
xlabel/ylabel	축 값과 단위 설명
title	그래프가 답하는 질문
legend	선·점·색상의 의미
grid	값을 읽는 보조선
tight_layout	제목과 축 글자 잘림 방지

크기, 선, 점, 범례와 축

```
plt.figure(figsize=(10, 6), dpi=120)
plt.scatter(
    X, y, s=60, alpha=0.7, marker="o",
    color="#3274a1", edgecolor="white",
    label="실제 데이터"
)
plt.plot(
    X, y_pred, color="#c44e52",
    linewidth=2, linestyle="--",
    label="예측 결과"
)
plt.title("실제 점수와 모델 예측",
        fontsize=15, fontweight="bold", pad=15)
plt.xlabel("공부 시간 (시간)", fontsize=11)
plt.ylabel("점수", fontsize=11)
plt.legend(loc="best", fontsize=10, frameon=True)
plt.grid(True, axis="y", linestyle="--", alpha=0.35)
plt.xlim(0, 10); plt.ylim(0, 100)
plt.xticks(rotation=0)
```

옵션	역할
s, linewidth	점과 선의 크기
alpha	투명도
marker, linestyle	점과 선의 모양
xlim, ylim	축 범위
xticks, yticks	눈금 위치와 회전

저장과 머신러닝 그래프 템플릿

여러 그래프와 저장

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].scatter(X, y); axes[0].set_title("원본 데이터")
axes[1].scatter(y_test, y_pred); axes[1].set_title("실제값과 예측값")
fig.tight_layout()
fig.savefig("reports/result.png", dpi=300, bbox_inches="tight")
plt.show()
```

잔차와 기준선

```
residuals = y_test - y_pred
plt.scatter(y_pred, residuals, alpha=.7)
plt.axhline(0, color="red", linestyle="--")
```

혼동 행렬

```
sns.heatmap(confusion_matrix(y_test, y_pred),
            annot=True, fmt="d", cmap="Blues")
plt.xlabel("예측 레이블"); plt.ylabel("실제 레이블")
```

Seaborn 공통 테마

```
sns.set_theme(style="whitegrid", palette="deep", font="Malgun Gothic")
```

시각적 정직성

축을 과도하게 잘라 차이를 과장하지 않고, 단위와 데이터가 학습용인지 시험용인지 제목이나 설명에 표시합니다. 저장은 `show()`보다 먼저 실행하는 편이 안전합니다.

프로젝트 완료 체크리스트

- ✓ 예측 시점, 타겟 y , 데이터 한 행의 의미를 문장으로 정의했다.
- ✓ 회귀/분류를 y 의 성격으로 선택했다.
- ✓ 예측 시점에 알 수 없는 열과 ID-타겟 누수 열을 제외했다.
- ✓ 결측치, 중복, 이상치, 타입과 단위를 확인했다.
- ✓ 전처리 전에 학습·시험 데이터를 분리했다.
- ✓ 전처리와 모델을 Pipeline으로 묶었다.
- ✓ 간단한 기준 모델보다 나은지 확인했다.
- ✓ 목적에 맞는 평가 지표를 선택했다.
- ✓ 교차검증으로 결과의 흔들림을 확인했다.
- ✓ 가장 크게 틀린 실제 사례를 분석했다.
- ✓ 시험 데이터는 모든 선택 후 마지막 평가에 사용했다.
- ✓ Pipeline 전체와 입력 스키마를 함께 저장했다.
- ✓ 데이터 기간, 코드, 라이브러리 버전, Seed와 결과를 기록했다.
- ✓ 모델의 한계와 사용하면 안 되는 상황을 문서화했다.

최종 원칙

좋은 머신러닝 프로젝트는 복잡한 모델보다 **명확한 문제**, **누수 없는 데이터 처리**, **공정한 평가**, **재현 가능한 기록**에서 시작합니다.